6

Métodos

OBJETIVOS

Neste capítulo você aprenderá:

- Como métodos e campos static são associados a uma classe inteira em vez de instâncias específicas da classe.
- Como utilizar métodos Math comuns disponíveis na API Java.
- Os mecanismos para passar informações entre métodos.
- Como o mecanismo de retorno/chamada de método é suportado pela pilha de chamadas de métodos e registros de ativação.
- Como pacotes agrupam classes relacionadas.
- Como utilizar a geração de números aleatórios para implementar aplicações de jogos de azar.
- Como a visibilidade das declarações é limitada a regiões específicas dos programas.
- O que é a sobrecarga de método e como criar métodos sobrecarregados.

Outline

6.1 Introdução

6.2 Módulos de programa em Java

6.3 Métodos `static`, campos `static` e classe `Math`

6.4 Declarando métodos com múltiplos parâmetros

6.5 Notas sobre a declaração e utilização de métodos

6.6 Pilha de chamadas de métodos e registros de ativação

6.7 Promoção e coerção de argumentos

6.8 Pacotes da API Java

6.9 Estudo de caso: geração de números aleatórios

- 6.9.1 Escalonamento e deslocamento generalizados de números aleatórios
- 6.9.2 Repetição de números aleatórios para teste e depuração

6.10 Estudo de caso: um jogo de azar (introdução a enumerações)

6.11 Escopo das declarações

6.12 Sobrecarga de método

6.1 Introdução

- **Técnica dividir para conquistar**
  - **Construir um programa grande a partir de pequenas partes (ou módulos)**
  - **Pode ser alcançado utilizando métodos**
- **Métodos `static` podem ser chamados sem a necessidade de um objeto da classe existir**
- **Geração de números aleatórios**
- **Constantes**

6.2 Módulos de programa em Java

- **Java Application Programming Interface (API)**
  - **Também conhecida como biblioteca de classes Java**
  - **Contém métodos e classes predefinidos**
    - **Classes relacionadas são agrupadas em pacotes**
    - **Inclui métodos para realizar cálculos matemáticos comuns, manipulações de caracteres/cadeia de caracteres, operações de entrada/saída, operações de banco de dados, operações de redes, processamento de arquivos, verificação de erros e muitas outras operações úteis**

Boa prática de programação 6.1

Familiarize-se com a rica coleção de classes e métodos fornecidos pela API do Java (`java.sun.com/javase/6/docs/api/index.html`). Na seção 6.8, será apresentada uma visão geral dos vários pacotes comuns. No Apêndice G, é explicado como navegar pela documentação da API do Java.

Observação de engenharia de software 6.1

Não tente reiventar a roda. Quando possível, reutilize as classes e métodos na API do Java. Isso reduz o tempo de desenvolvimento de programas e evita a introdução de erros.

6.2 Módulos de programa em Java (Cont.)

- **Métodos**
  - **Denominados funções ou procedimentos em outras linguagens**
  - **Modularizam programas separando suas tarefas em unidades autocontidas**
  - **Permitem a abordagem dividir para conquistar**
  - **São reutilizados em novos programas**
  - **Evitar repetição de código**

Observação de Engenharia de Software 6.2

Para promover a capacidade de reutilização de software, todos os métodos devem estar limitados à realização de uma única tarefa bem definida e o nome do método deve expressar essa tarefa efetivamente. Esses métodos tornam mais fácil escrever, depurar, manter e modificar programas.

Dica de prevenção de erros 6.1

Um pequeno método que realiza uma tarefa é mais fácil de testar e depurar do que um método maior que realiza muitas tarefas.

Observação de engenharia de software 6.3

Se você não puder escolher um nome conciso que expresse a tarefa de um método, seu método talvez tente realizar um número excessivo de tarefas. Em geral, é melhor dividir esse método em várias declarações de métodos menores.

6.3 Métodos `static`, campos `static` e a Classe Math

- **Método `static` (ou método de classe)**
  - **Aplicado à classe em que é declarado como um todo ao invés de um objeto específico da classe**
  - **Chamada de um método `static`:**
    ***NomeDaClasse* . *nomeDoMetodo* (*argumentos*)**
  - **Todos os métodos da classe `Math` são `static`**
    - **exemplo: `Math.sqrt( 900.0 )`**

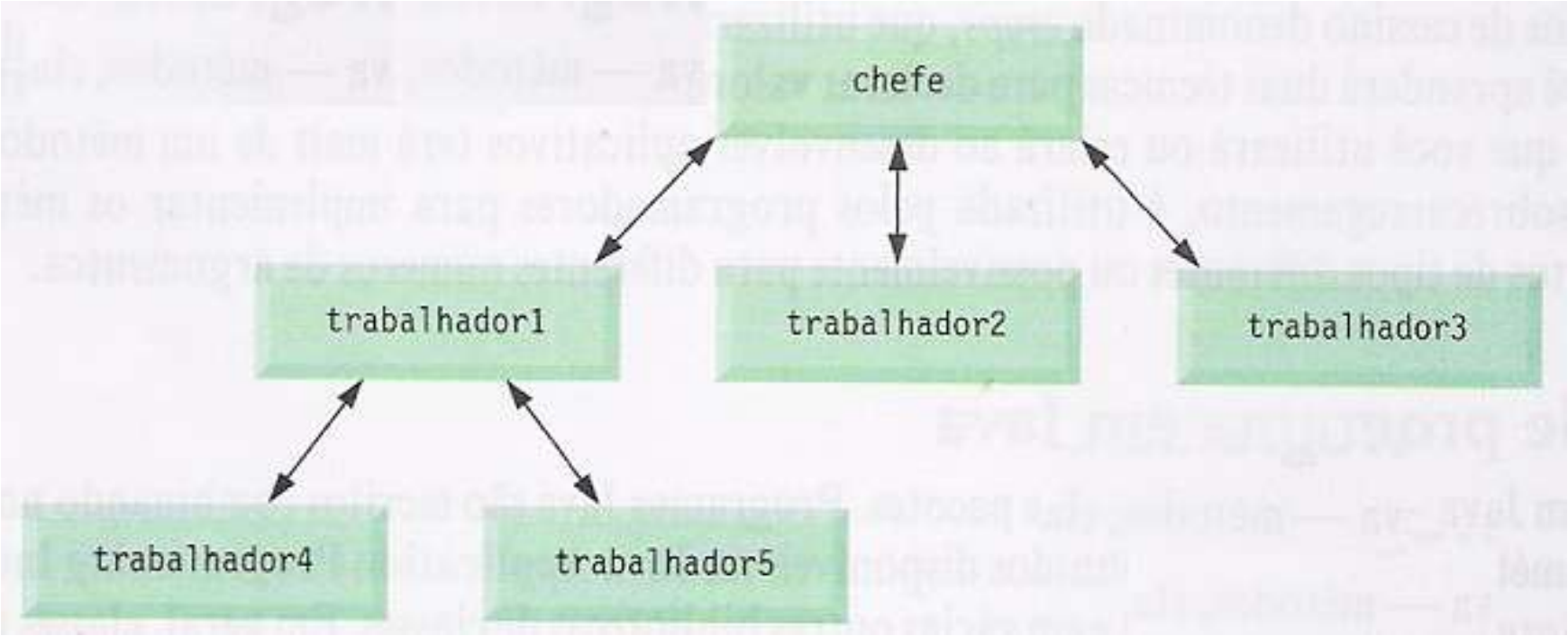

Fig. 6.1 | Relacionamento hierárquico de método trabalhador/método chefe.

Observação de engenharia de software 6.4

A classe `Math` faz parte do pacote `java.lang`, que é implicitamente importado pelo compilador, assim não é necessário importar a classe Math para utilizar seus métodos.

6.3 Métodos `static`, campos `static` e a Classe Math (Cont.)

- **Constantes**
  - **Palavra-chave `final`**
  - **Não podem ser alteradas após sua inicialização**
- **Campos `static` (ou variáveis de classe)**
  - **São campos onde uma cópia da variável é compartilhada entre todos os objetos da classe**
- **`Math.PI` e `Math.E` são campos `final static` da classe `Math`**

Método	Descrição	Exemplo
`abs( x )`	valor absoluto *x*	`abs(23.7)` é 23.7 `abs(0.0)` é 0.0 `abs(-23.7)` é 23.7
`ceil( x )`	arredonda *x* para o menor inteiro não menor que *x*	`ceil(9.2)` é 10.0 `ceil(-9.8)` é -9.0
`cos( x )`	co-seno trigonométrico de *x* (*x* em radianos)	`cos(0.0)` é 1.0
`exp( x )`	método exponencial e^x	`exp(1.0)` é 2.71828 `exp(2.0)` é 7.38906
`floor( x )`	arredonda *x* para o maior inteiro não maior que *x*	`floor(9.2)` é 9.0 `floor(-9.8)` é -10.0
`log( x )`	logaritmo natural de *x* (base *e*)	`log(Math.E)` é 1.0 `log(Math.E * Math.E)` é 2.0
`max( x, y )`	maior valor de *x* e *y*	`max(2.3, 12.7)` é 12.7 `max(-2.3, -12.7)` é -2.3
`min( x, y )`	menor valor de *x* e *y*	`min( 2.3, 12.7 )` é 2.3 `min( -2.3, -12.7 )` é -12.7
`pow( x, y )`	*x* elevado à potência de *y* (i.e., x^y)	`pow( 2.0, 7.0 )` é 128.0 `pow( 9.0, 0.5 )` é 3.0
`sin( x )`	seno trigonométrico de *x* (*x* em radianos)	`sin( 0.0 )` é 0.0
`sqrt( x )`	raiz quadrada de *x*	`sqrt( 900.0 )` é 30.0
`tan( x )`	tangente trigonométrica de *x* (*x* em radianos)	`tan( 0.0 )` é 0.0

Fig. 6.2 | Métodos da classe Math.

6.3 Métodos `static`, campos `static` e a Classe Math (Cont.)

- **Método `main`**
  - **`main` é declarado `static` então pode ser invocado sem a criação de um objeto da classe que contém o `main`**
  - **Qualquer classe pode conter um método `main`**
    - **A JVM invoca o método `main` pertencente a classe especificada pelo primeiro argumento da linha de comando do comando `java`**

6.4 Declarando métodos com múltiplos parâmetros

- **Múltiplos parâmetros podem ser declarados em uma lista separada por vírgula.**
  - **Argumentos passados em uma chamada de método devem ser consistentes com números, tipos e ordem dos parâmetros**
    - **Geralmente chamados de parâmetros formais**

```java
// Fig. 6.3: MaximumFinder.java
// Método maximum declarado pelo programador.
import java.util.Scanner;

public class MaximumFinder
{
   // obtém 3 valores de ponto flutuante e localiza o valor máximo
   public void determineMaximum()
   {
      // cria Scanner para entrada de dados
      Scanner input = new Scanner( System.in );

      // obtém entrada do usuário
      System.out.print(
         "Digite 3 valores de ponto-flutuante separados por espaço: " );
      double number1 = input.nextDouble(); // lê primeiro double
      double number2 = input.nextDouble(); // lê segundo double
      double number3 = input.nextDouble(); // lê terceiro double

      // determina o valor máximo
      double result = maximum( number1, number2, number3 );

      // exibe o valor máximo
      System.out.println( "Máximo é: " + result );
   } // fim do método determineMaximum

```

```
27     // Retorna o máximo de três parâmetros
28     public double maximum( double x, double y, double z )
29     {
30        double maximumValue = x; // supõe que x seja o maior inicial
31
32        // determina se y é maior que maximumValue
33        if ( y > maximumValue )
34           maximumValue = y;
35
36        // determina se z é maior que maximumValue
37        if ( z > maximumValue )
38           maximumValue = z;
39
40        return maximumValue;
41     } // fim do método maximum
42 } // fim da classe MaximumFinder
```

```java
// Fig. 6.4: MaximumFinderTest.java
// Applicativo para testar a classe MaximumFinder.

public class MaximumFinderTest
{
   // ponto de partida do aplicativo
   public static void main( String args[] )
   {
      MaximumFinder maximumFinder = new MaximumFinder();
      maximumFinder.determineMaximum();
   } // fim do main
} // fim da classe MaximumFinderTest
```

```
Digite 3 valores de ponto-flutuante separados por espaço: 9.35 2.74 5.1
Máximo é: 9.35
```

```
Digite 3 valores de ponto-flutuante separados por espaço: 5.8 12.45 8.32
Máximo é: 12.45
```

```
Digite 3 valores de ponto-flutuante separados por espaço: 6.46 4.12 10.54
Máximo é: 10.54
```

Erro comum de programação 6.1

Declarar parâmetros de método do mesmo tipo como `float x, y` ao invés de `float x, float y` é um erro de sintaxe – um tipo é requerido para cada parâmetro na lista de parâmetros.

Observação de engenharia de software 6.5

Um método com muitos parâmetros pode estar realizando tarefas demais. Considere dividir o método em métodos menores que realizam tarefas separadas. Como uma diretriz, se possível, tente ajustar o cabeçalho do método em uma linha.

6.4 Declarando métodos com múltiplos parâmetros (Cont.)

- **Reutilizando o método `Math.max`**
  - **A expressão `Math.max( x, Math.max( y, z ) )` determina o maior dos argumentos `y` e `z`, após isso determina o mairo entre `x` e o novo valor**
- **Concatenação de strings**
  - **Utilizando o operador `+` com duas `Strings` há a concatenação delas em uma nova `String`**
  - **Utilizando o operador `+` com uma `String` e um valor de outro tipo de dado há a concatenação de `String` com a representação de `String` do dado**
    - **Quando outro valor é um objeto, seu método `toString` é chamado para gerar sua representação de `String`**

Erro comum de programação 6.2

É um erro de sintaxe quebrar uma literal `String` em múltiplas linhas dentro de um programa. Se uma String não couber em uma linha, divida a String em várias Strings menores e utilize a concatenação para formar a String desejada.

Erro comum de programação 6.3

Confundir o operador + utilizado para concatenação de String com o operador + utilizado para adição pode levar a resultados estranhos. Java avalia os operandos de um operador da esquerda para a direita. Por exemplo, suponha que a variável inteira `y` tem o valor `5`, a expressão `"y + 2 = " + y + 2` resulta na String `"y + 2 = 52"`, não em `"y + 2 = 7"`, porque o primeiro valor de `y` (5) é concatenado com a String `"y + 2 = "`, em seguida o valor 2 é concatenado com a nova e maior String `"y + 2 = 5"`. A expressão `"y + 2 = " + (y + 2)` produz o resultado desejado `"y + 2 = 7"`.

6.5 Notas sobre a declaração e utilização de métodos

- **Há três maneiras de chamar um método:**
  - **Utilizando o próprio nome do método para chamar um outro método da mesma classe**
  - **Utilizando uma variável que contém uma referência a um objeto, seguido por um ponto (.) e pelo nome do método para chamar um método do objeto referenciado**
  - **Utilizando o nome da classe e um ponto (.) para chamar um método `static` de uma classe**
- **Métodos `static` não podem chamar membros não-`static` da mesma classe diretamente**

6.5 Notas sobre a declaração e utilização de métodos (Cont.)

- **Há três maneiras de retornar o controle à instrução que chama o método:**
  - **Se o método não retornar um resultado:**
    - **Fluxo do programa alcança chave direita de fechamento do método ou**
    - **Programa executa a instrução `return;`**
  - **Se o método retornar um resultado:**
    - **Programa executas a instrução `return` *expressão* ;**
      - ***expressão* é primeiro avaliada e então seu valor é retornado ao chamador**

Erro comum de programação 6.4

Declarar um método fora do corpo de uma declaração de classe ou dentro do corpo de um outro método é um erro de sintaxe.

Erro comum de programação 6.5

Omitir o tipo-do-valor-de-retorno em uma declaração de método é um erro de sintaxe.

Erro comum de programação 6.6

Colocar um ponto-e-vírgula após o parêntese direito que envolve a lista de parâmetros de uma declaração de método é um erro de sintaxe.

Erro comum de programação 6.7

Redeclarar um parâmetro de método como uma variável local no corpo do método é um erro de compilação.

Erro comum de programação 6.8

Esquecer de retornar um valor em um método que deve retornar um valor é um erro de compilação. Se um tipo do valor de retorno além de `void` for especificado, o método deverá conter uma instrução `return` que retorne um valor consistente com o tipo-do-valor-de-retorno do método. Retornar um valor de um método cujo tipo de retorno foi declarado como `void` é um erro de compilação.

6.6 Pilha de chamadas de método e registros de ativação

- **Pilhas**
  - **Estruturas de dados do tipo último a entrar, primeiro a sair (Last-in, first-out -- LIFO)**
    - **Items são colocados no topo (empilhados)**
    - **Items são removidos do topo (desempilhados)**
- **Pilha de execução de programas**
  - **Também chamado de pilha de chamadas de método**
  - **Retorna endereços de chamadas de métodos empilhados na pilha quando eles chamam outros métodos e desempilhados quando o controle é retornado aos respectivos métodos chamadores**

6.6 Pilha de chamadas de método e registros de ativação (Cont.)

- **Variáveis locais de métodos são armazenadas em uma porção desta pilha conhecida como registros de ativação ou quadros de pilha**
  - **Quando a última variável referenciando um certo objeto é retirada da pilha, o objeto não poderá mais ser acessado pelo programa**
    - **Será apagado da memória durante a "coleta de lixo"**
  - **<u>Estouro de pilha</u> ocorre quando a pilha não consegue alocar espaço suficiente para um registro de ativação de método**

6.7 Promoção e coerção de argumentos

- **Promoção de argumentos**
  - **Java promoverá um argumento de método para o tipo que o método espera de acordo com regras de promoção**
  - **Valores em uma expressão são promovidos ao tipo "mais alto" em uma expressão (uma cópia temporária do valor é criada)**
  - **Conversão de valores em tipos menores resultará em erro de compilação, a menos que o programador force explicitamente a conversão**
    - **Coloque o tipo de dado desejado entre parênteses antes do valor**
      - **Exemplo: `(int) 4.5`**

Tipo	Promoções válidas
`double`	Nenhuma
`float`	`double`
`long`	`float` ou `double`
`int`	`long`, `float` ou `double`
`char`	`int`, `long`, `float` ou `double`
`short`	`int`, `long`, `float` ou `double` (mas não `char`)
`byte`	`short`, `int`, `long`, `float` ou `double` (mas não `char`)
`boolean`	Nenhuma (valores `boolean` não são considerados números em Java)

Fig. 6.5 | **Promoções permitidas para tipos primitivos.**

Erro comum de programação 6.9

Converter um valor de tipo primitivo em um outro tipo primitivo pode alterar o valor se o novo tipo não for uma promoção válida. Por exemplo, converter um valor de ponto flutuante em um valor integral pode introduzir erros de truncamento (perda da parte fracionária) no resultado.

6.8 Pacotes da API Java

- **A inclusão da declaração**
  `import java.util.Scanner;`
  permite ao programador (a) utilizar `Scanner` **em vez de** `java.util.Scanner`
- **Documentação da Java API**
  - **java.sun.com/javase/6/docs/api/index.html**
- **Overview of packages in JDK 6.0**
  - **java.sun.com/javase/6/docs/api/overview-summary.html**

Pacote	Descrição
`java.applet`	O Java Applet Package contém uma classe e várias interfaces exigidas para criar applets Java — programas que executam nos navegadores da Web. (Os applets serão discutidos no Capítulo 20, Introdução a applets Java; e as interfaces no Capítulo 10, Programação orientada a objetos: Polimorfismo.)
`java.awt`	O Java Abstract Window Toolkit Package contém as classes e interfaces exigidas para criar e manipular GUIs no Java 1.0 e 1.1. Nas versões atuais do Java, os componentes GUI Swing dos pacotes `javax.swing` são freqüentemente utilizados em seu lugar. (Alguns elementos do pacote `java.awt` serão discutidos no Capítulo 11, Componentes GUI: Parte 1, Capítulo 12, Imagens gráficas e Java2D e Capítulo 22, Componentes GUI: Parte 2.)
`java.awt.event`	O Java Abstract Window Toolkit Event Package contém classes e interfaces que permitem o tratamento de eventos para componentes GUI tanto nos pacotes `java.awt` como `javax.swing`. (Você aprenderá mais sobre esse pacote no Capítulo 11, Componentes GUI: Parte 1 e no Capítulo 22, Componentes GUI: Parte 2.)
`java.io`	O Java Input/Output Package contém classes e interfaces que permitem aos programas gerar entrada e saída de dados. (Você aprenderá mais sobre esse pacote no Capítulo 14, Arquivos e fluxos.)
`java.lang`	O Java Language Package contém classes e interfaces (discutidas por todo esse texto) que são exigidas por muitos programas Java. Esse pacote é importado pelo compilador para todos os programas, assim o programador não precisa fazer isso.

Fig. 6.6 | Java API packages (a subset). (Part 1 of 2)

`java.net`	O Java Networking Package contém classes e interfaces que permitem aos programas comunicar-se via redes de computadores, como a Internet. (Você verá mais detalhes sobre isso no Capítulo 24, Redes.)
`java.text`	O Java Text Package contém classes e interfaces que permitem aos programas manipular números, datas, caracteres e strings. O pacote fornece recursos de internacionalização que permitem a um programa ser personalizado para um local específico (por exemplo, um programa pode exibir strings em diferentes idiomas com base no país do usuário).
`java.util`	O Java Utilities Package contém classes utilitárias e interfaces que permitem ações como manipulações de data e hora, processamento de números aleatórios (classe `Random`), armazenamento e processamento de grandes volumes de dados e a divisão de strings em parte menores chamadas tokens (classe `StringTokenizer`). (Você aprenderá mais sobre os recursos desse pacote no Capítulo 19, Coleções.)
`javax.swing`	O Java Swing GUI Components Package contém classes e interfaces para componentes GUI Swing do Java que fornecem suporte para GUIs portáveis. (Você aprenderá mais sobre esse pacote no Capítulo 11: Componentes GUI: Parte 1 e no Capítulo 22: Componentes GUI: Parte 2.)
`javax.swing.event`	O Java Swing Event Package contém classes e interfaces que permitem o tratamento de eventos (por exemplo, responder a cliques de botão) para componentes GUI no pacote `javax.swing`. (Você aprenderá mais sobre esse pacote no Capítulo 11: Componentes GUI: Parte 1 e no Capítulo 22: Componentes GUI: Parte 2.)

Fig. 6.6 | Java API packages (a subset). (Part 2 of 2)

Boa prática de programação 6.2

A documentação on-line da API do Java é fácil de pesquisar e fornece vários detalhes sobre cada classe. À medida que você aprende uma classe, deve criar o hábito de examinar a classe na documentação on-line para informações adicionais.

6.9 Estudo de caso: Geração de números aleatórios

- **Geração de números aleatórios**
  - **Método `static` `random` da classe `Math`**
    - **Retorna valores `double` no intervalo `0.0` <= x < `1.0`**
  - **Classe `Random` do pacote `java.util`**
    - **Pode produzir valores pseudo-aleatórios `boolean`, `byte`, `float`, `double`, `int`, `long` e valores Gaussianos**
    - **É semeado com o horário do dia atual para gerar diferentes seqüências de números aleatórios cada vez que o programa executa**

```java
// Fig. 6.7: RandomIntegers.java
// Inteiros aleatórios deslocados e escalonados.
import java.util.Random; // programa usa a classe Random

public class RandomIntegers
{
   public static void main( String args[] )
   {
      Random randomNumbers = new Random(); // gerador de número aleatório
      int face; // armazena cada inteiro aleatório gerado

      // executa laço 20 vezes
      for ( int counter = 1; counter <= 20; counter++ )
      {
         // seleciona inteiro aleatório de 1 a 6
         face = 1 + randomNumbers.nextInt( 6 );

         System.out.printf( "%d  ", face ); // exibe o valor gerado

         // se counter for divisível por 5, pula uma linha
         if ( counter % 5 == 0 )
            System.out.println();
      } // fim do for
   } // fim do main
} // fim da classe RandomIntegers
```

```
1   5   3   6   2
5   2   6   5   2
4   4   4   2   6
3   1   6   2   2
```

```
6   5   4   2   6
1   2   5   1   3
6   3   2   2   1
6   4   2   6   4
```

```java
// Fig. 6.8: RollDie.java
// Roda um dado de seis lados 6000 vezes.
import java.util.Random;

public class RollDie
{
   public static void main( String args[] )
   {
      Random randomNumbers = new Random(); // gerador de número aleatório

      int frequency1 = 0; // contador para 1s lançados
      int frequency2 = 0; // contador para 2s lançados
      int frequency3 = 0; // contador para 3s lançados
      int frequency4 = 0; // contador para 4s lançados
      int frequency5 = 0; // contador para 5s lançados
      int frequency6 = 0; // contador para 6s lançados

```

```
18      int face; // armazena o valor lançado mais recentemente
19
20      // resume os resultados de 6000 lançamentos de um dado
21      for ( int roll = 1; roll <= 6000; roll++ )
22      {
23         face = 1 + randomNumbers.nextInt( 6 ); // número de 1 a 6
24
25         // determina o valor de lançamento e incrementa o contador
adequado
26         switch ( face )
27         {
28            case 1:
29               ++frequency1; // incrementa o contador de 1s
30               break;
31            case 2:
32               ++frequency2; // incrementa o contador de 2s
33               break;
34            case 3:
35               ++frequency3; // incrementa o contador de 3s
36               break;
37            case 4:
38               ++frequency4; // incrementa o contador de 4s
39               break;
40            case 5:
41               ++frequency5; // incrementa o contador de 5s
42               break;
43            case 6:
44               ++frequency6; // incrementa o contador de 6s
45               break; // opcional ao fim do switch
46         } // fim do switch
47      } // fim do for
48
```

```java
49        System.out.println( "Face\tFrequência" ); // saída
50        System.out.printf( "1\t%d\n2\t%d\n3\t%d\n4\t%d\n5\t%d\n6\t%d\n",
51           frequency1, frequency2, frequency3, frequency4,
52           frequency5, frequency6 );
53     } // fim do main
54 } // fim da classe RollDie
```

```
Face    Frequência
1       982
2       1001
3       1015
4       1005
5       1009
6       988
```

```
Face    Frequência
1       1029
2       994
3       1017
4       1007
5       972
6       981
```

6.9.1 Escalonamento e deslocamento generalizados de números aleatórios

- **Geração aleatória em um certo intervalo ou seqüência**
  - **Utilize a expressão** *valorDeDeslocamento* **+** *diferençaEntreValores* ***** `randomNumbers.nextInt(` *fatorDeEscalonamento* `)` **onde:**
    - *valorDeDeslocamento* **é o primeiro número no intervalo desejado de valores**
    - *diferençaEntreValores* **representa a diferença entre números consecutivos na seqüência**
    - *fatorDeEscalonamento* **especifica quantos números estão no intervalo**

6.9.2 Repetição de números aleatórios para teste e depuração

- **Para obter um objeto `Random` para gerar a mesma seqüência de números aleatórios a cada execução do programa, isto é, toda vez que o programa executa, semeie o objeto com um determinado valor**
  - **Quando criar um objeto `Random`:**
    ```
    Random randomNumbers =
      new Random( seedValue );
    ```
  - **Utilize o método `setSeed`:**
    ```
    randomNumbers.setSeed( seedValue );
    ```
  - **`seedValue` deve ser um argumento do tipo `long`**

Dica de prevenção de erros 6.2

Enquanto um programa estiver sob desenvolvimento, crie um objeto Random com um valor específico de semente para produzir uma seqüência repetível de números aleatórios toda vez que o programa é executado. Se ocorrer um erro de lógica, corrija esse erro e teste o programa novamente com o mesmo valor de semente – isso permite reconstruir a mesma seqüência de números aleatórios que causou o erro. Depois que os erros de lógica forem removidos, crie o objeto Random sem utilizar um valor de semente, fazendo com que o objeto Random gere uma nova seqüência de números aleatórios toda vez que o programa é executado.

```java
// Fig. 6.9: Craps.java
// A classe Craps simula o jogo de dados craps.
import java.util.Random;

public class Craps
{
   // cria um gerador de números aleatórios para uso no método rollDice
   private Random randomNumbers = new Random();

   // enumeração com constantes que representam o status do jogo
   private enum Status { CONTINUE, WON, LOST };

   // constantes que representam lançamentos comuns dos dados
   private final static int SNAKE_EYES = 2;
   private final static int TREY = 3;
   private final static int SEVEN = 7;
   private final static int YO_LEVEN = 11;
   private final static int BOX_CARS = 12;

```

```
20    // joga uma partida de craps
21    public void play()
22    {
23       int myPoint = 0; // pontos se não ganhar ou perder na 1ª
rolagem
24       Status gameStatus; // pode conter CONTINUE, WON ou LOST
25
26       int sumOfDice = rollDice(); // 1a rolagem dos dados
27
28       // determina status do jogo e pontuação baseado no 1o
lançamento
29       switch ( sumOfDice )
30       {
31          case SEVEN: // ganha com 7 no primeiro lançamento
32          case YO_LEVEN: // ganha com 11 no primeiro lançamento
33             gameStatus = Status.WON;
34             break;
35          case SNAKE_EYES: // perde com 2 no primeiro lançamento
36          case TREY: // perde com 3 no primeiro lançamento
37          case BOX_CARS: // perde com 12 no primeiro lançamento
38             gameStatus = Status.LOST;
39             break;
40          default: // não ganhou nem perdeu, registra os pontos
41             gameStatus = Status.CONTINUE; // jogo não acabou
42             myPoint = sumOfDice; // informa a pontuação
43             System.out.printf( "Ponto é %d\n", myPoint );
44             break; // opcional ao fim do switch
45       } // fim do switch
46
```

```
47        // enquanto o jogo não estiver completo
48        while ( gameStatus == Status.CONTINUE ) // nem WON nem LOST
49        {
50           sumOfDice = rollDice(); // lança os dados novamente
51
52           // determina status do jogo
53           if ( sumOfDice == myPoint ) // vitória por pontuação
54              gameStatus = Status.WON;
55           else
56              if ( sumOfDice == SEVEN ) // perde obtendo 7 antes de atingir
a pontuação
57                 gameStatus = Status.LOST;
58        } // fim do while
59
60        // exibe mensagem de vitória ou derrota
61        if ( gameStatus == Status.WON )
62           System.out.println( "Jogador vence" );
63        else
64           System.out.println( "Jogador perde" );
65     } // fim do método play
66
```

```
67    // lança os dados, calcula a soma e mostra os resultados
68    public int rollDice()
69    {
70       // seleciona valores aleatórios do dado
71       int die1 = 1 + randomNumbers.nextInt( 6 ); // primeiro lançamento
72       int die2 = 1 + randomNumbers.nextInt( 6 ); // segundo lançamento
73
74       int sum = die1 + die2; // soma dos valores dos dados
75
76       // exibe resultados desse lançamento
77       System.out.printf( "Jogador lançou %d + %d = %d\n",
78          die1, die2, sum );
79
80       return sum; // retorna a soma dos dados
81    } // fim do método rollDice
82 } // fim da classe Craps
```

```java
// Fig. 6.10: CrapsTest.java
// Aplicativo para testar a classe Craps.

public class CrapsTest
{
   public static void main( String args[] )
   {
      Craps game = new Craps();
      game.play(); // joga uma partida de craps
   } // fim do main
} // fim da classe classe CrapsTest
```

```
Jogador lançou 5 + 6 = 11
Jogador vence
```

```
Jogador lançou 1 + 2 = 3
Jogador perde
```

```
Jogador lançou 5 + 4 = 9
Ponto é 9
Jogador lançou 2 + 2 = 4
Jogador lançou 2 + 6 = 8
Jogador lançou 4 + 2 = 6
Jogador lançou 3 + 6 = 9
Jogador vence
```

```
Jogador lançou 2 + 6 = 8
Ponto é 8
Jogador lançou 5 + 1 = 6
Jogador lançou 2 + 1 = 3
Jogador lançou 1 + 6 = 7
Jogador perde
```

6.10 Estudo de caso: um jogo de azar (introdução a enumerações)

- **Enumerações**
  - **Tipos declarados pelo programador consistindo de um conjunto de constantes**
  - **Palavra-chave `enum`**
  - **Um nome do tipo (e.g. `Status`)**
  - **Constantes de enumeração (e.g. `WON`, `LOST` e `CONTINUE`)**
    - **Não podem ser comparadas com `ints`**

Boa prática de programação 6.3

Utilize somente letras maiúsculas nos nomes de constantes. Isso faz com que as constantes sejam destacadas em um programa e lembrem o programador de que constantes de enumeração não são variáveis.

Boa prática de programação 6.4

Utilizar constantes de enumeração (como Status.WON, Status.LOST e Status.CONTINUE) em vez de valores literais inteiros (como 0, 1 e 2) pode tornar os programas mais fáceis de ler e manter.

6.11 Escopo de Declarações

- **Regras básicas de escopo**
  - **O escopo de uma declaração de parâmetro é o corpo do método em que a declaração aparece**
  - **O escopo de uma declaração de variável local vai do ponto em que a declaração aparece até o fim desse bloco**
  - **O escopo de uma declaração de variável local que aparece na seção de inicialização do cabeçalho de uma instrução `for` é o corpo da instrução `for` com as outras expressões no cabeçalho**
  - **O escopo de um método ou campo de uma classe é o corpo inteiro da classe**

6.11 Escopo de Declarações (Cont.)

- **Sombreamento**
  - **Um campo é dito sombreado (ou oculto) se a variável local ou parâmetro possuir o mesmo nome do campo**
    - **Até que a variável local ou parâmetro saia do escopo (bloco termina a execução)**

Erro comum de programação 6.10

Um erro de compilação ocorre quando uma variável local é declarada mais de uma vez em um método.

Dica de prevenção de erros 6.3

Utilize nomes diferentes para campos e variáveis locais para ajudar a evitar erros de lógica sutis que ocorrem quando um método é chamado e uma variável local do método sombreia um campo com o mesmo nome da classe.

```java
// Fig. 6.11: Scope.java
// Classe que demonstra escopos de campo e de variável local.

public class Scope
{
   // campo acessível para todos os métodos dessa classe
   private int x = 1;

   // método begin cria e inicializa a variável local x
   // e chama os métodos useLocalVariable e useField
   public void begin()
   {
      int x = 5; // variável local x do método sombreia o campo x

      System.out.printf( "local x no método begin é %d\n", x );

      useLocalVariable(); // useLocalVariable tem uma variável local x
      useField(); // useField usa campo x da classe Scope
      useLocalVariable(); // useLocalVariable reinicializa var. local x
      useField(); // campo x da classe Scope retém seu valor

```

```
22         System.out.printf( "\nlocal x no método begin é %d\n", x );
23      } // fim do método begin
24
25      // cria e inicializa a var. local x durante cada chamada
26      public void useLocalVariable()
27      {
28         int x = 25; // inicializda toda vez que useLocalVariable é chamado
29
30         System.out.printf(
31            "\nlocal x ao entrar no método useLocalVariable é %d\n", x );
32         ++x; // modifies this method's local variable x
33         System.out.printf(
34            "local x antes de sair do método useLocalVariable é %d\n", x );
35      } // fim do método useLocalVariable
36
37      // modifica o campo x da classe Scope durante cada chamada
38      public void useField()
39      {
40         System.out.printf(
41            "\ncampo x ao entrar no método useField é %d\n", x );
42         x *= 10; // modifica o campo x da classe Scope
43         System.out.printf(
44            "campo x antes de sair do método useField é %d\n", x );
45      } // fim do método useField
46   } // fim da classe Scope
```

```java
// Fig. 6.12: ScopeTest.java
// Aplicativo para testar a classe Scope.

public class ScopeTest
{
   // ponto de partida do aplicativo
   public static void main( String args[] )
   {
      Scope testScope = new Scope();
      testScope.begin();
   } // fim do main
} // fim da classe ScopeTest
```

```
local x no método begin é 5

local x ao entrar no método useLocalVariable é 25
local x antes de sair do método useLocalVariable é 26

campo x ao entrar no método useField é 1
campo x antes de sair do método useField é 10

local x ao entrar no método useLocalVariable é 25
local x antes de sair do método useLocalVariable é 26

campo x ao entrar no método useField é 10
campo x antes de sair do método useField é 100

local x no método begin é 5
```

6.12 Sobrecarga de método

- **Sobrecarga de método**
  - **Múltiplos métodos contendo mesmo nome, mas diferentes tipos, número ou ordem de parâmetros em sua lista de parâmetros**
  - **O compilador decide qual o método está sendo chamado de acordo com a lista de argumentos passada na chamada do método em um dos métodos sobrecarregados**
    - **Um nome de método e número, tipo e ordem de seus parâmetros formam sua assinatura**
  - **Diferenças em tipo de retorno são irrelevantes na sobrecarga de método**
    - **Métodos sobrecarregados podem possuir diferentes tipos de retorno**
    - **Métodos com diferentes tipos de retorno mas mesma assinatura causam um erro de compilação**

```java
// Fig. 6.13: MethodOverload.java
// Declarações de métodos sobrecarregados.

public class MethodOverload
{
   // teste de métodos square sobrecarregados
   public void testOverloadedMethods()
   {
      System.out.printf( "Quadrado do inteiro 7 é %d\n", square( 7 ) );
     System.out.printf( "Quadrado do double 7.5 é %f\n", square( 7.5 ) );
   } //  fim do método testOverloadedMethods

   // método square com argumento int
   public int square( int intValue )
   {
      System.out.printf( "\nChamado square com argumento int: %d\n",
         intValue );
      return intValue * intValue;
   } // fim do método square com argumento int

   // método square argumento double
   public double square( double doubleValue )
   {
      System.out.printf( "\nChamado square com argumento double: %f\n",
         doubleValue );
      return doubleValue * doubleValue;
   } // fim do método square com argumento double
} // fim da classe MethodOverload
```

```java
// Fig. 6.14: MethodOverloadTest.java
// Aplicativo para testar a classe MethodOverload.

public class MethodOverloadTest
{
   public static void main( String args[] )
   {
      MethodOverload methodOverload = new MethodOverload();
      methodOverload.testOverloadedMethods();
   } // fim do main
} // fim da classe MethodOverloadTest
```

```
Chamado square argumento int: 7
Quadrado do inteiro 7 é 49

Chamado square com argumento double: 7.500000
Quadrado do double 7.5 é 56.250000
```

```java
// Fig. 6.15: MethodOverloadError.java
// Métodos sobrecarregados com assinaturas idênticas
// resulta em erros de compilação, mesmo com tipos de retorno diferentes.

public class MethodOverloadError
{
   // declaração do método square com argumento int
   public int square( int x )
   {
      return x * x;
   }

   // segunda declaração do método square com arguemnto int
   // causa erro de compilação mesmo com tipos de retorno diferentes
   public double square( int y )
   {
      return y * y;
   }
} // fim da classe MethodOverloadError
```

```
MethodOverloadError.java:15: square(int) is already defined in
MethodOverloadError
   public double square( int y )
                 ^
1 error
```

Erro comum de programação 6.11

Declarar métodos sobrecarregados com listas de parâmetros idênticas é um erro de compilação independente de os tipos de retorno serem diferentes.

Referências

Adaptado de: “Teaching Resources for Java How to Program, 6/e”.

Deitel & Deitel. Java como programar, 6 ed.

Kathy Sierra e Bert Bates. Head First Java!.

Notas de aula da disciplina Projeto com Orientação a Objetos.